UNIVERSITY OF FRONTIER TECHNOLOGY, BANGLADESH

Department of Cyber Security Engineering

Lab Report

Name of The Experiment: Implementation of CRUD Operations using Laravel Framework

Course Code : PROG 212

Course Title : Android & Web Application Development Sessional

Submitted by,

Ragib Shahriar Abeg
ID: 2304017
Level -2, Term – 1
Session: 2023-24

Submitted to,

Md. Abdullah,
Lecturer,
Department of Cyber Security Engineering

Experiment Name

Implementation of CRUD Operations using Laravel Framework

Objective

The objective of this experiment is to learn and implement the four fundamental database operations — Create, Read, Update, and Delete (CRUD) — using the Laravel PHP framework. Laravel is a modern, open-source web application framework that follows the MVC (Model-View-Controller) architectural pattern.

Another objective is to understand how Laravel organizes a web application through its routing system, Eloquent ORM, Blade templating engine, and Artisan command-line tool by building a fully functional student management system from scratch.

Theory

Laravel is a free, open-source PHP web framework designed for building modern web applications. It provides an expressive and elegant syntax that simplifies common web development tasks such as routing, authentication, database interaction, and session management.

Laravel follows the MVC (Model-View-Controller) design pattern which separates the application logic into three interconnected components:

- Model – Represents the data and business logic. It interacts directly with the database using Laravel's Eloquent ORM (Object Relational Mapper).
- View – Represents the user interface. Laravel uses the Blade templating engine to render HTML pages dynamically.
- Controller – Acts as the bridge between the Model and View. It handles incoming HTTP requests, processes data, and returns appropriate responses.

CRUD stands for Create, Read, Update, and Delete — the four basic operations performed on any database. In a web application, these operations correspond to specific HTTP methods:

- Create → POST request → Saves new data to the database
- Read → GET request → Retrieves and displays data from the database
- Update → PUT/PATCH request → Modifies existing data in the database
- Delete → DELETE request → Removes data from the database

Laravel's Route::resource() method provides a convenient way to define all CRUD routes in a single line, automatically mapping HTTP verbs to the corresponding controller methods.

List of Topics for this Experiment

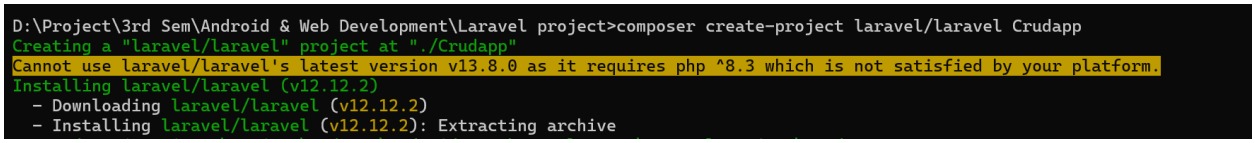
- Laravel Installation and Project Setup – Creating a new Laravel project using Composer.
- Database Configuration – Connecting Laravel to a MySQL database via the .env file.
- Migration – Creating and running database migrations to define table structure.
- Model – Creating an Eloquent Model to interact with the students table.
- Controller – Building a StudentController with all CRUD methods.
- Routes – Registering resource routes in web.php to handle all HTTP requests.
- Blade Views – Creating index, create, and edit views using the Blade templating engine.
- CRUD Testing – Verifying all four operations work correctly in the browser.

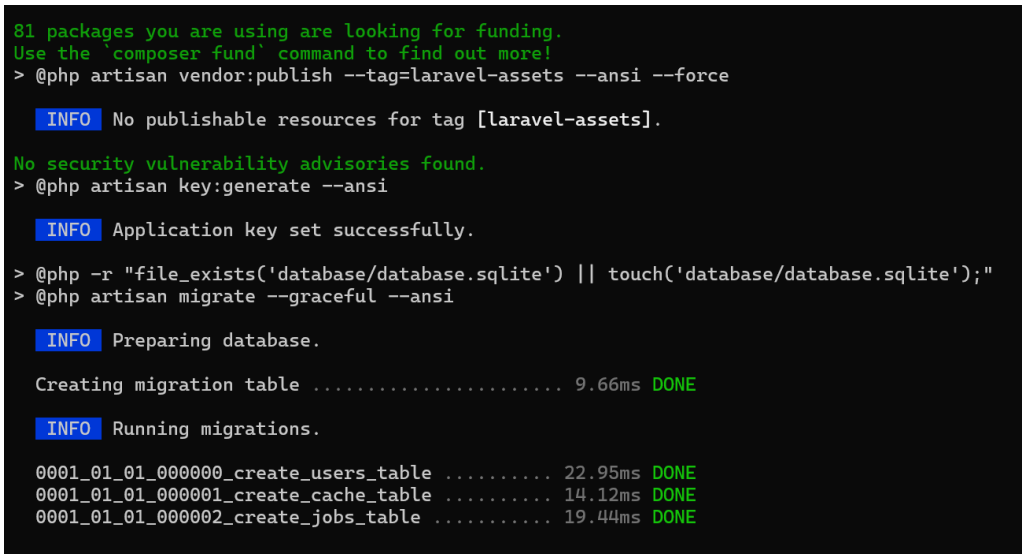
Program / Implementation

Step 1: Project Setup

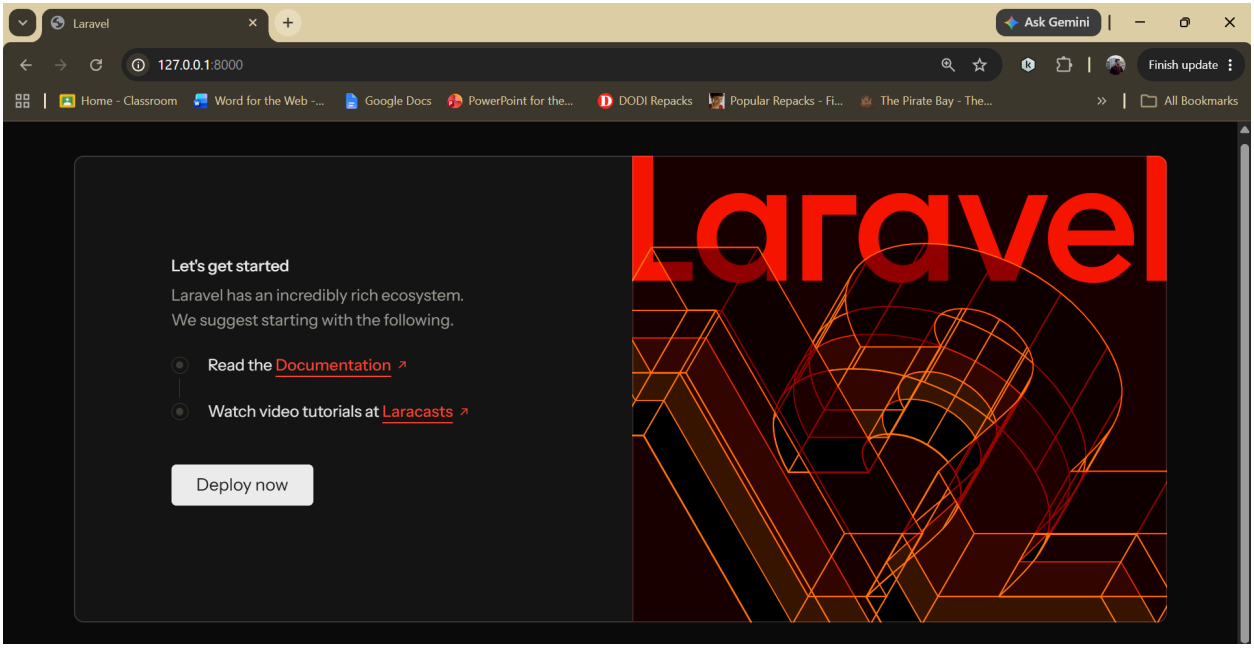
A new Laravel project named Crudapp was created using Composer. The development server was started using the php artisan serve command.

```
composer create-project laravel/laravel Crudapp
cd Crudapp
php artisan serve
```





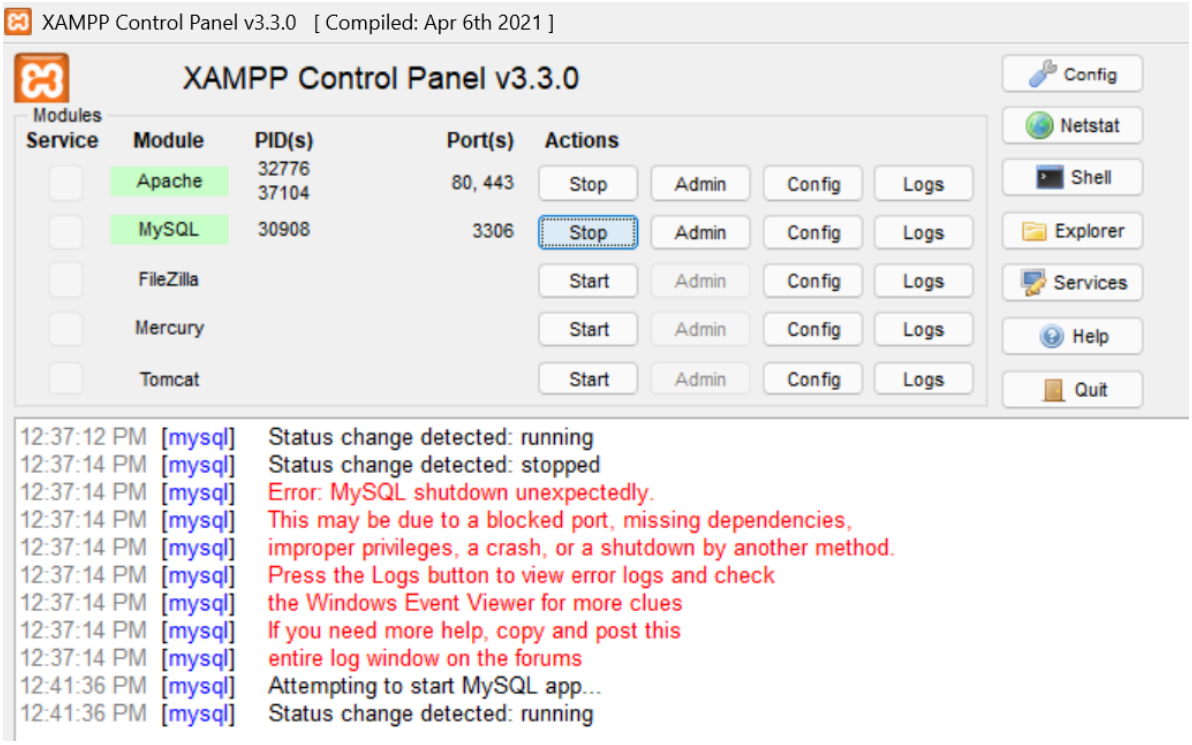
```
PS D:\Project\3rd Sem\Android & Web Development\Laravel project\
Crudapp> php artisan serve

  INFO  Server running on [http://127.0.0.1:8000].

  Press Ctrl+C to stop the server

  2026-06-09 23:08:27 / ..... ~ 512.28ms
  2026-06-09 23:08:27 /favicon.ico ..... ~ 510.46ms
```

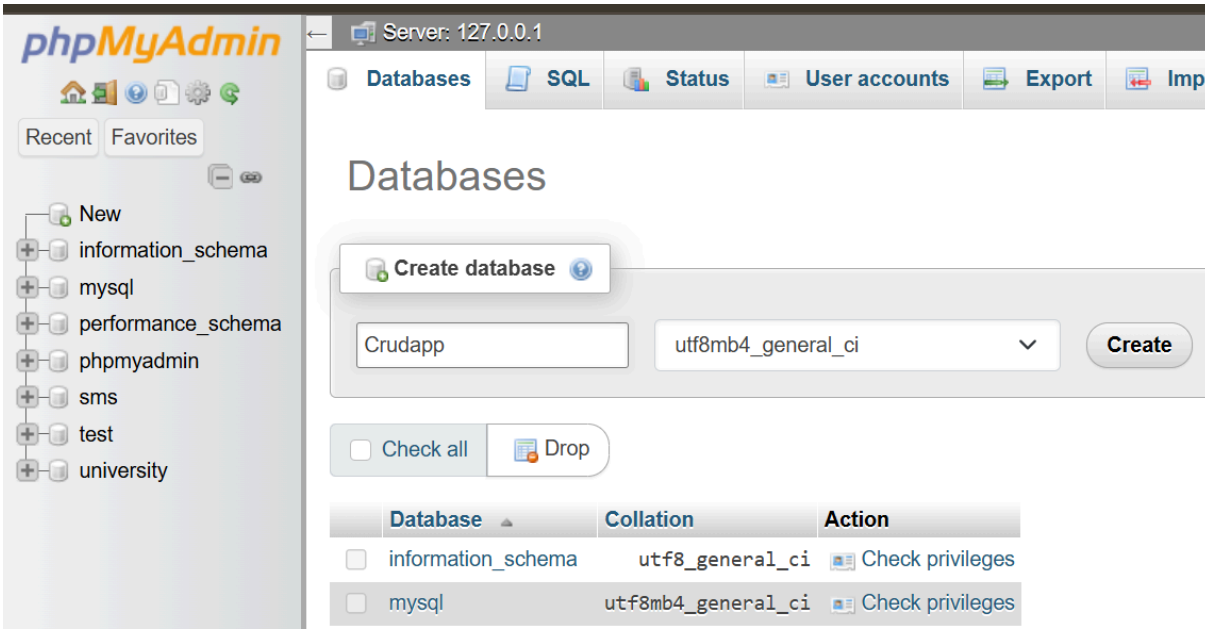


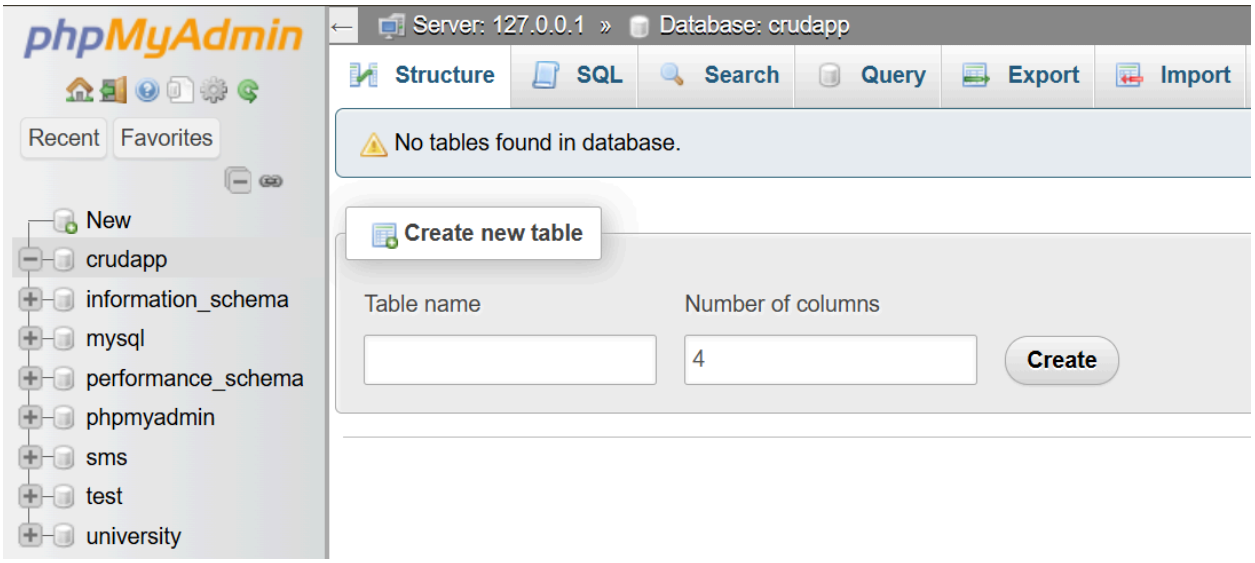
Step 2: Database Configuration

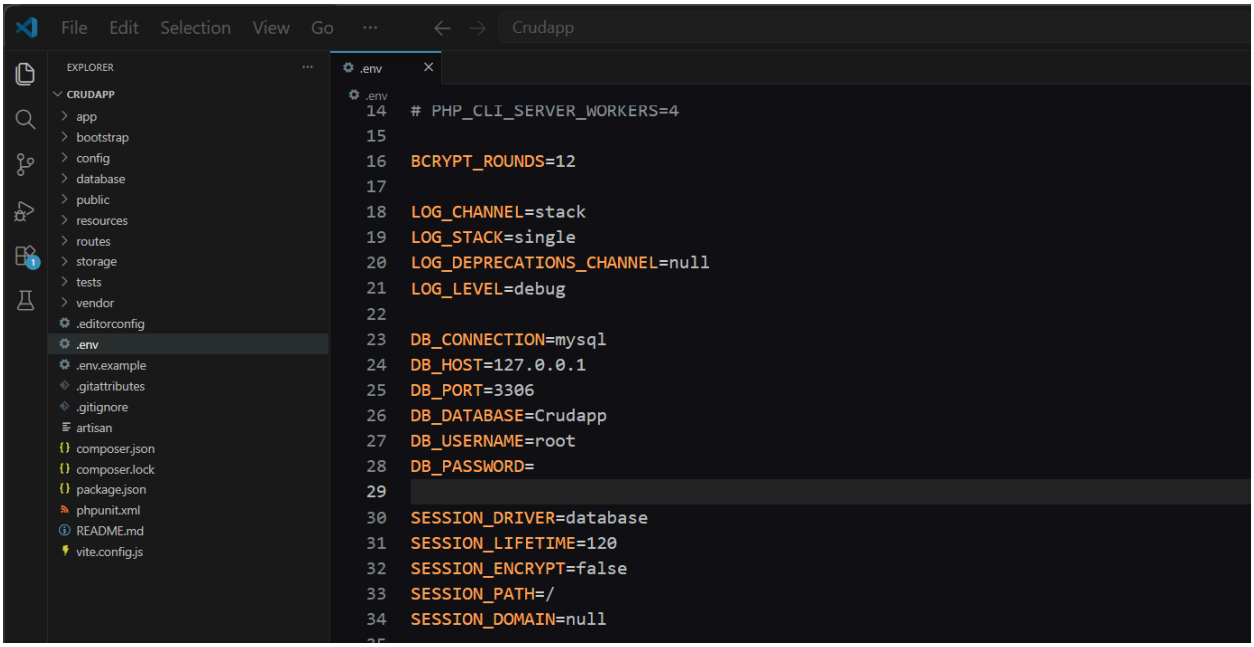
A MySQL database named Crudapp was created using phpMyAdmin. The .env file was updated with the following database credentials to establish the connection:

```
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=Crudapp
DB_USERNAME=root
DB_PASSWORD=
```






phpMyAdmin
Recent
Favorites
New
crudapp
information_schema
mysql
performance_schema
phpmyadmin
sms
test
university
Server: 127.0.0.1 » Database: crudapp
Structure
SQL
Search
Query
Export
Import
No tables found in database.
Create new table
Table name
Number of columns
4
Create

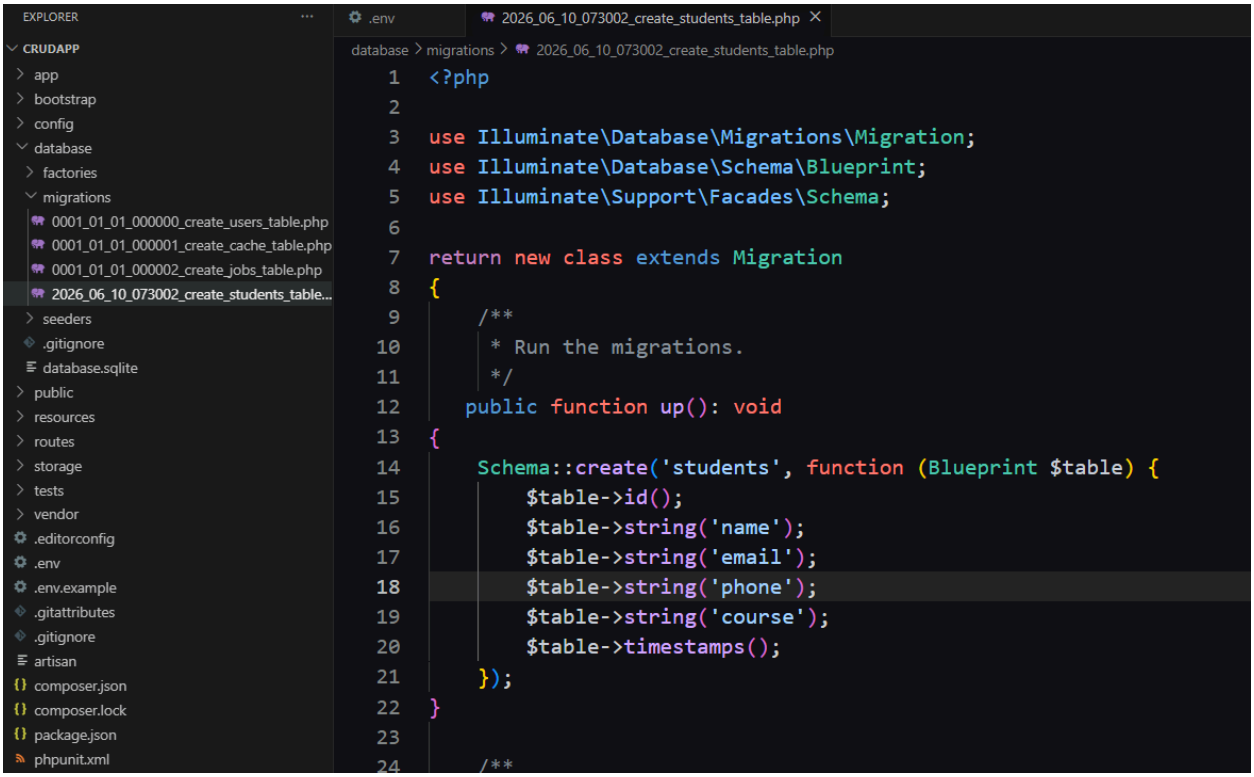

File Edit Selection View Go
Crudapp
EXPLORER
CRUDAPP
app
bootstrap
config
database
public
resources
routes
storage
tests
vendor
.editorconfig
.env
.env.example
.gitattributes
.gitignore
artisan
composer.json
composer.lock
package.json
phpunit.xml
README.md
vite.config.js
.env
# PHP_CLI_SERVER_WORKERS=4
BCRYPT_ROUNDS=12
LOG_CHANNEL=stack
LOG_STACK=single
LOG_DEPRECATIONS_CHANNEL=null
LOG_LEVEL=debug
DB_CONNECTION=mysql
DB_HOST=127.0.0.1
DB_PORT=3306
DB_DATABASE=Crudapp
DB_USERNAME=root
DB_PASSWORD=
SESSION_DRIVER=database
SESSION_LIFETIME=120
SESSION_ENCRYPT=false
SESSION_PATH=/
SESSION_DOMAIN=null

Step 3: Migration

A migration file was created to define the structure of the students table. The migration was then executed to create the table in the database.

```
php artisan make:migration create_students_table
```

```
D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp>php artisan make:migration create_students_table

  INFO  Migration [D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp\database\migrations\2026_06_10_073002_create
_students_table.php] created successfully.
```



The migration file was edited to define the following columns: id, name, email, phone, course, and timestamps. The migration was then run:

```
php artisan migrate
```

```
D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp>php artisan migrate

  INFO  Running migrations.

  2026_06_10_073002_create_students_table ..... 16.06ms DONE
```

Table	Action	Rows	Type	Collation	Size	Overhead
cache	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
cache_locks	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
failed_jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
jobs	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
job_batches	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
migrations	Browse Structure Search Insert Empty Drop	4	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
password_reset_tokens	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
sessions	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	48.0 KiB	-
students	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	16.0 KiB	-
users	Browse Structure Search Insert Empty Drop	0	InnoDB	utf8mb4_unicode_ci	32.0 KiB	-
10 tables	Sum	4	InnoDB	utf8mb4_general_ci	272.0 KiB	0 B

Step 4: Model

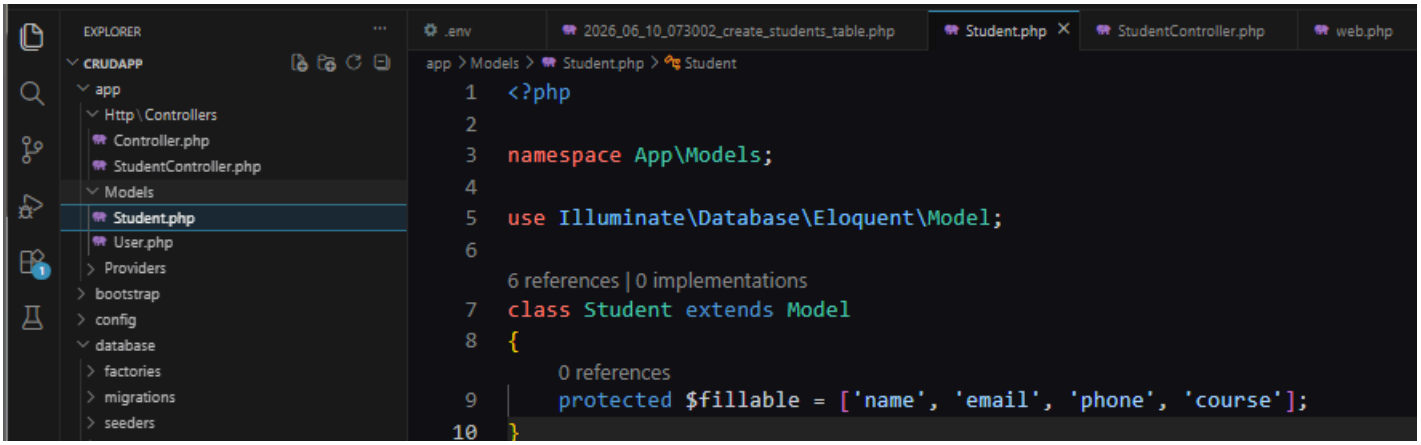
A Student Model was created using Artisan. The $fillable property was defined to specify which columns can be mass-assigned by user input, which is required by Laravel's mass assignment protection.

```
php artisan make:model Student
```

```
D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp>php artisan make:model Student

  INFO  Model [D:\Project\3rd Sem\Android & Web Development\Laravel project\Crudapp\app\Models\Student.php] created successfully.
```

Inside Student.php:

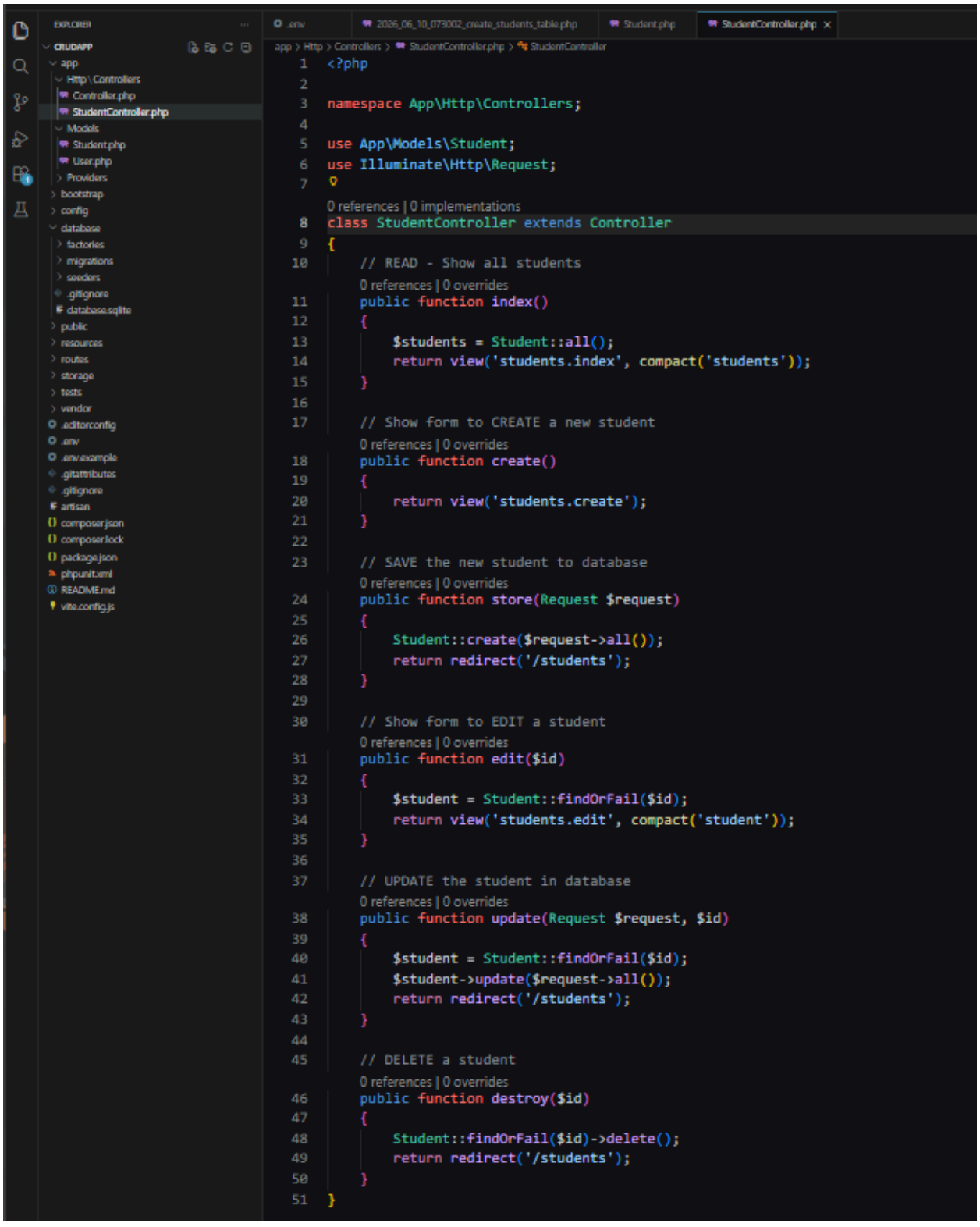
```
protected $fillable = ['name', 'email', 'phone', 'course'];
```



Step 5: Controller

A StudentController was created containing all six CRUD methods: index(), create(), store(), edit(), update(), and destroy(). Each method handles a specific operation on the students table.

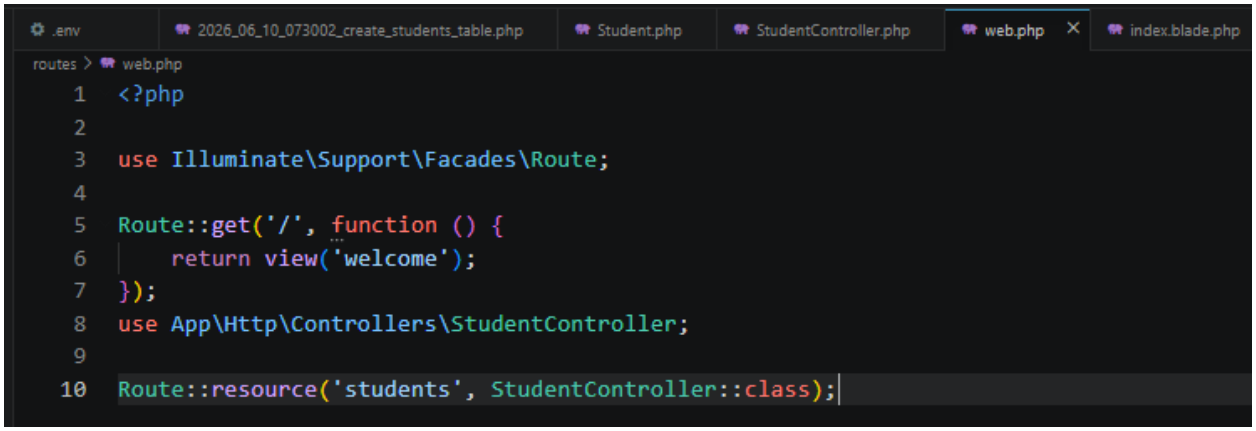
```
php artisan make:controller StudentController
```



Step 6: Routes

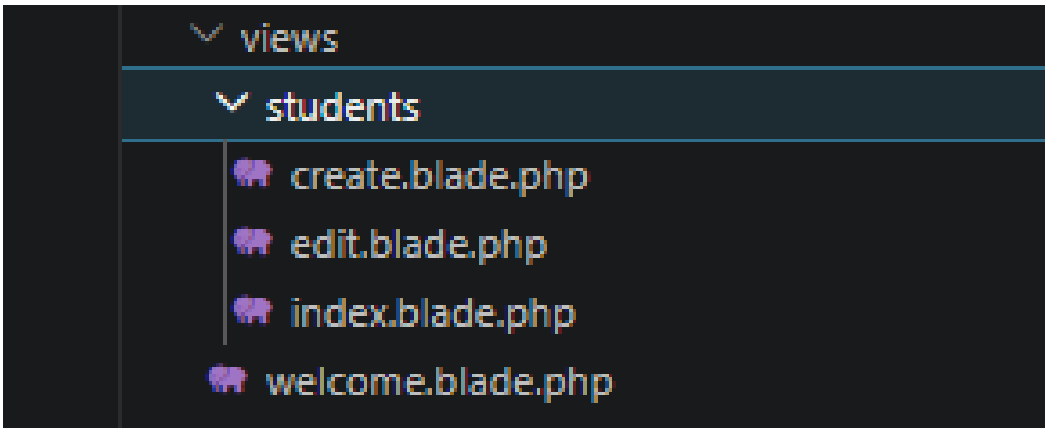
A single resource route was registered in routes/web.php that automatically generates all required CRUD routes — eliminating the need to define each route individually.

```
Route::resource('students', StudentController::class);
```

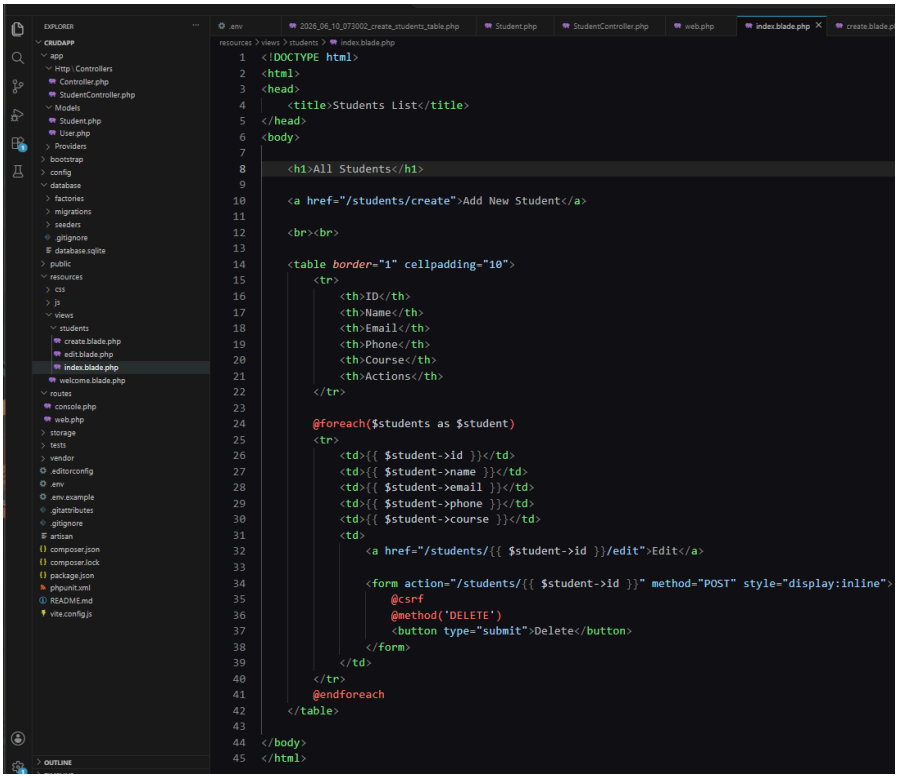


Step 7: Blade Views

Three Blade view files were created inside the resources/views/students/ directory:

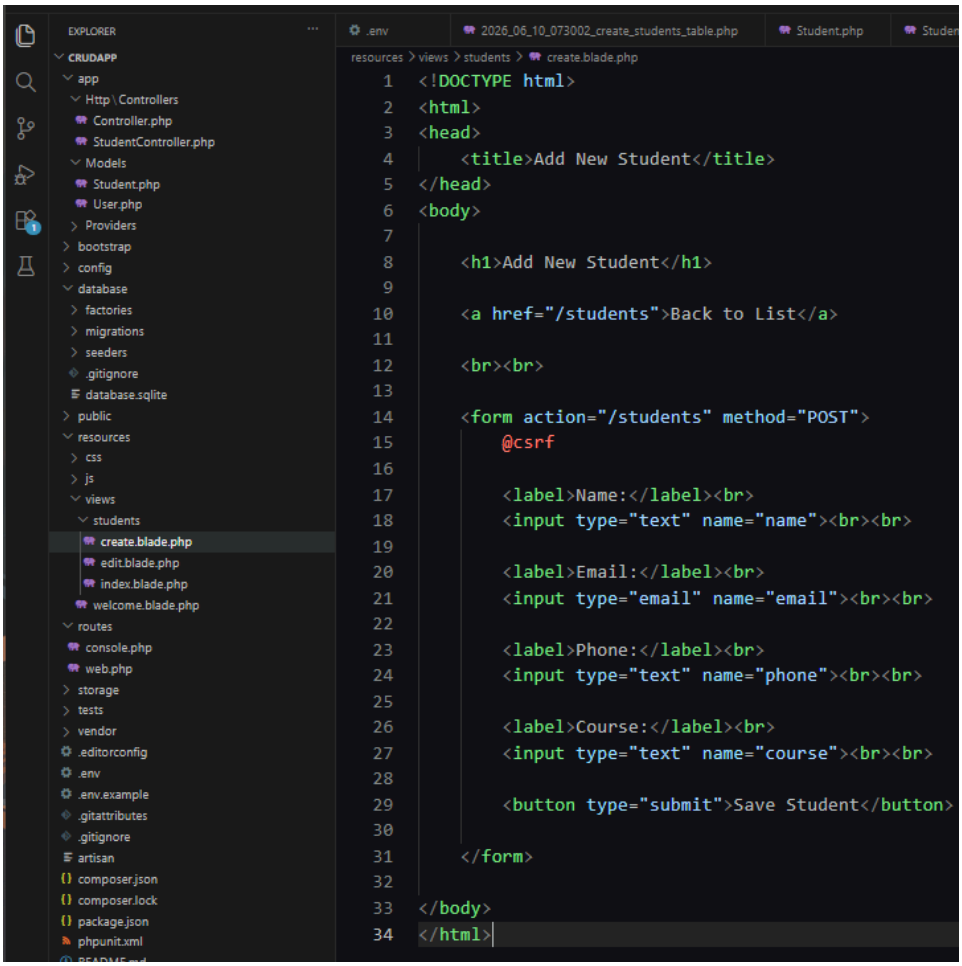


- index.blade.php – Displays all students in a table with Edit and Delete buttons.



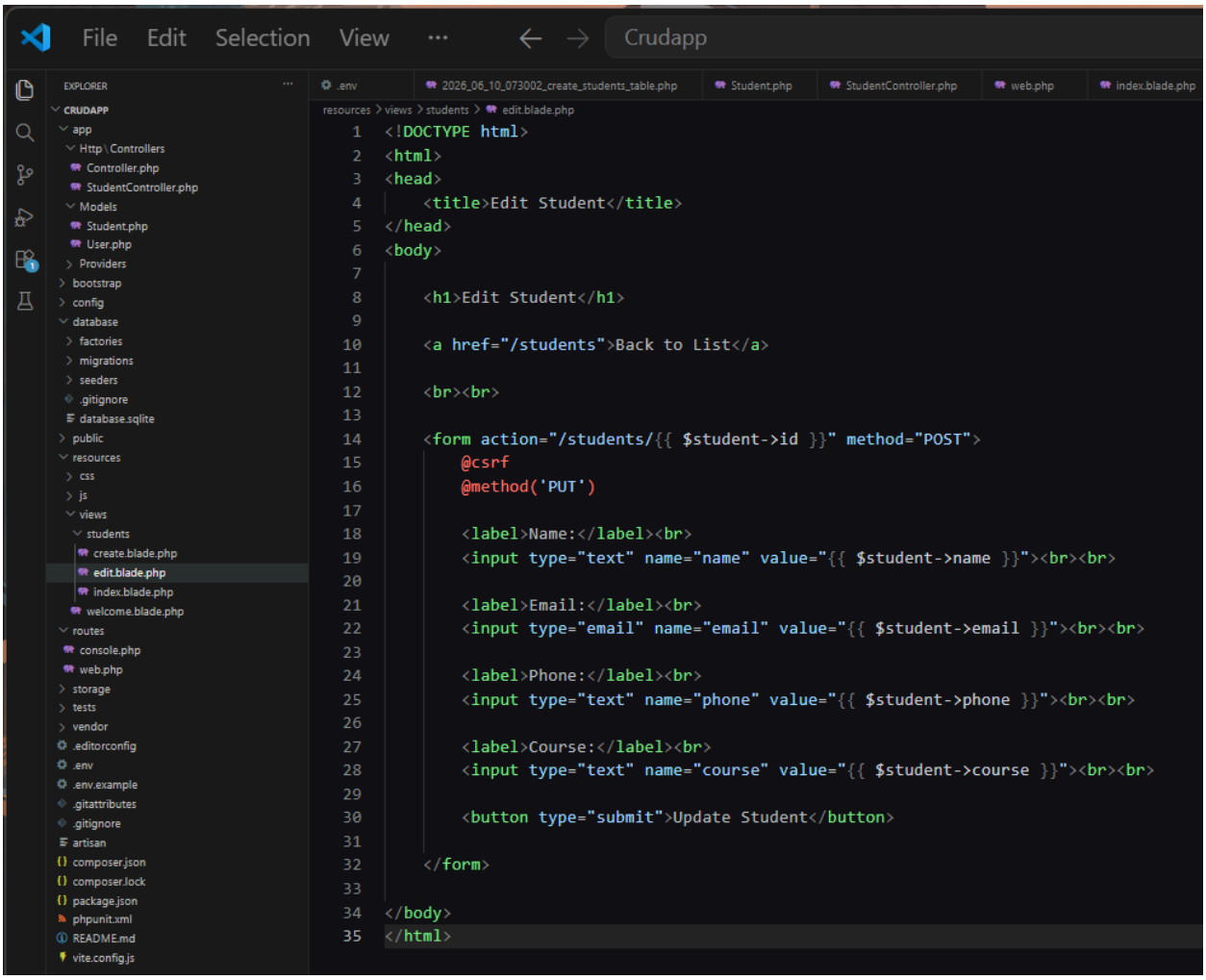
```blade
<!DOCTYPE html>
<html>
<head>
    <title>Students List</title>
</head>
<body>

    <h1>All Students</h1>

    <a href="/students/create">Add New Student</a>

    <br><br>

    <table border="1" cellpadding="10">
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Email</th>
            <th>Phone</th>
            <th>Course</th>
            <th>Actions</th>
        </tr>

        @foreach($students as $student)
        <tr>
            <td>{{ $student->id }}</td>
            <td>{{ $student->name }}</td>
            <td>{{ $student->email }}</td>
            <td>{{ $student->phone }}</td>
            <td>{{ $student->course }}</td>
            <td>
                <a href="/students/{{ $student->id }}/edit">Edit</a>

                <form action="/students/{{ $student->id }}" method="POST" style="display:inline">
                    @csrf
                    @method('DELETE')
                    <button type="submit">Delete</button>
                </form>
            </td>
        </tr>
        @endforeach
    </table>

</body>
</html>
```

- create.blade.php – A form with Name, Email, Phone, and Course fields to add a new student.

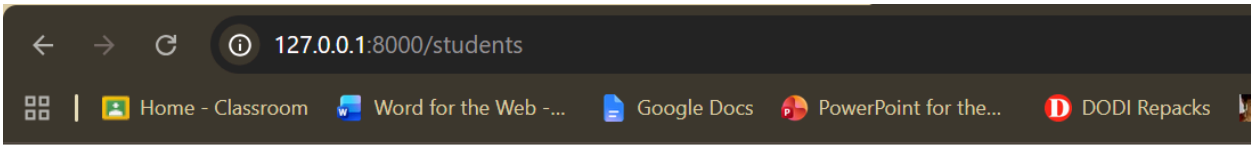


```blade
<!DOCTYPE html>
<html>
<head>
    <title>Add New Student</title>
</head>
<body>

    <h1>Add New Student</h1>

    <a href="/students">Back to List</a>

    <br><br>

    <form action="/students" method="POST">
        @csrf

        <label>Name:</label><br>
        <input type="text" name="name"><br><br>

        <label>Email:</label><br>
        <input type="email" name="email"><br><br>

        <label>Phone:</label><br>
        <input type="text" name="phone"><br><br>

        <label>Course:</label><br>
        <input type="text" name="course"><br><br>

        <button type="submit">Save Student</button>

    </form>

</body>
</html>
```

- edit.blade.php – A pre-filled form that allows editing of an existing student's information.



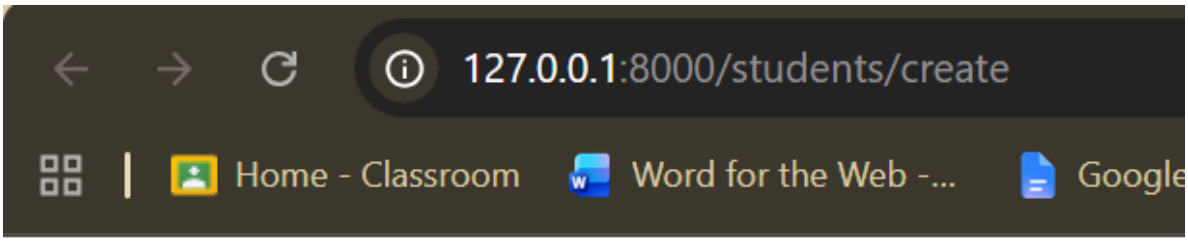
```
<!DOCTYPE html>
<html>
<head>
    <title>Edit Student</title>
</head>
<body>

    <h1>Edit Student</h1>

    <a href="/students">Back to List</a>

    <br><br>

    <form action="/students/{{ $student->id }}" method="POST">
        @csrf
        @method('PUT')

        <label>Name:</label><br>
        <input type="text" name="name" value="{{ $student->name }}"><br><br>

        <label>Email:</label><br>
        <input type="email" name="email" value="{{ $student->email }}"><br><br>

        <label>Phone:</label><br>
        <input type="text" name="phone" value="{{ $student->phone }}"><br><br>

        <label>Course:</label><br>
        <input type="text" name="course" value="{{ $student->course }}"><br><br>

        <button type="submit">Update Student</button>

    </form>

</body>
</html>
```



All Students

Add New Student

ID	Name	Email	Phone	Course	Actions



Add New Student

Back to List

Name:

Email:

Phone:

Course:

Save Student

Add New Student

Back to List

Name:
Ragib Shahriar Abeg

Email:
ragibabeg1@gmail.com

Phone:
01602086423

Course:
Web and Android Developm

Save Student

All Students

Add New Student

ID	Name	Email	Phone	Course	Actions
1	Ragib Shahriar Abeg	ragibabeg1@gmail.com	01602086423	Web and Android Development	Edit Delete

Edit Student

Back to List

Name:
Ragib Shahriar Abeg

Email:
ragibabeg1@gmail.com

Phone:
01602086423

Course:
Operating System

Update Student

Practical Applications

- Used to build real-world data management systems such as student portals, employee management systems, and inventory systems.
- Demonstrates how web applications interact with databases to store, retrieve, modify, and remove records.
- Laravel's MVC pattern promotes clean, maintainable, and scalable code structure suitable for professional projects.
- The Blade templating engine simplifies dynamic HTML rendering, making front-end integration faster and cleaner.
- Resource routing reduces code repetition and follows RESTful API design conventions widely used in industry.
- Migrations ensure database version control, allowing teams to track and reproduce database structure changes.

Discussion

During this experiment, a complete CRUD web application was built step by step using the Laravel framework. Starting from project creation and database configuration, the application was developed through migrations, model creation, controller logic, route registration, and Blade view design.

All four CRUD operations were successfully tested in the browser. A new student record was created using the Add New Student form and saved to the MySQL database. The record was then displayed on the All Students list page. The Edit button opened a pre-filled form where the course field was modified and updated in the database. Finally, the Delete button removed the record from the database and redirected back to the list page.

The experiment demonstrated how Laravel's built-in tools such as Eloquent ORM, Artisan commands, resource routes, and Blade templates work together to simplify and speed up web application development significantly.

Conclusion

This experiment successfully demonstrated the implementation of CRUD operations using the Laravel PHP framework. By building a Student Management System from scratch, a practical understanding of Laravel's MVC architecture, database migration, Eloquent ORM, RESTful routing, and Blade templating was gained.

The knowledge acquired from this experiment forms a strong foundation for developing more complex and feature-rich web applications in future projects involving Laravel and other modern PHP framework.